Postmortem: Slowloris starved legit clients at the connection cap

Giselle Wu, CS60 Lab 2, Fall 2026
Drafted with assistance from Claude (Anthropic); all numbers, bytes and log lines are from my own test runs.

What failed

My first hardened server was thread-per-connection with every per-request defense in place: a 5 s whole-request deadline (2 s when busy), size limits, and strict framing. It also had a fixed cap of `MAX_CONNECTIONS = 500` and answered `503` once the cap was reached. Every single-request attack passed. My red-team flood (`redteam/floods.py`) still took it down for legit users.

Attack: 600 attacker connections. Each one sends this one byte every 0.5 s and never sends the blank line that ends the headers:

```
GET /helloworld.html HTTP/1.1\r\nHost: x\r\nX-a: bbbbbbbbbb...   (no \r\n\r\n, ever)
```

When the server drops a connection, the attacker reconnects within ~50 ms. Meanwhile a legit client sends `GET /helloworld.html HTTP/1.1` with `Connection: close` every 0.2 s.

Expected: every legit request gets `HTTP/1.1 200 OK` (the assignment's "keeps serving well-behaved clients throughout the attack").

Observed: the legit client got `200` on only **47 of 88** requests. The other 41 failed. The bytes a failing legit client received:

```
HTTP/1.1 503 Service Unavailable\r\nContent-Type: text/plain\r\nContent-Length: 24\r\nConne
→ ConnectionResetError(54, 'Connection reset by peer')   (body never arrived)
```

The server log shows the deadline doing its job: the attackers were being dropped constantly (1,844 times in 20 s):

```
('127.0.0.1', 57761): -> 408 (client too slow)
('127.0.0.1', 57759): -> 408 (client too slow)
('127.0.0.1', 57755): -> 408 (client too slow)
```

So the server didn't hang or crash, and every slow connection was cut off on schedule. Legit users were still locked out.

Root cause

The defense worked one connection at a time, but the attack targeted the shared pool of slots. In the accept loop:

```
with active_lock:
    full = active_connections >= MAX_CONNECTIONS
    if not full:
        active_connections += 1
if full:
    send_error(client_socket, 503)
    client_socket.close()
```

With 600 attackers and 500 slots, the server is always full. A slot frees only when an attacker hits its deadline (2 s under load). The 100 attackers waiting outside reconnect within ~50 ms, while the legit client arrives at most every 200 ms. Whoever's SYN lands right after a slot frees gets it, and there are 100 attackers retrying for every legit request, so the legit client mostly loses. The timeout limits how long one attacker can hold a slot, but nothing limits how quickly attackers take slots back. Thread-per-connection makes this worse, because each slot is a real thread and file descriptor, so the cap can't simply be raised forever.

A **second bug** shows up in the captured bytes: the 503 never arrived intact. The main thread called `close()` while the client's request was still unread in the kernel buffer. TCP then sends a RST instead of a FIN, and the RST throws away data the client hadn't read yet. So even the rejection looked like a crash to the client.

Fix

1. **Evict instead of refusing.** Each thread records when it starts waiting for a request in a shared `waiting` dict. When the server is full, the accept loop picks the connection that has waited longest and calls `shutdown(SHUT_RDWR)` on it. That wakes its thread's `recv()`, and the thread exits. The new connection gets the slot. A real client finishes in milliseconds and a Slowloris client never finishes, so the longest waiter is almost always an attacker. A `503` is now sent only when every slot is busy *sending* a response.
2. **Clean 503.** Before closing, half-close (`SHUT_WR`) and do one non-blocking `recv()` to drain the request, so `close()` sends a FIN instead of a RST.
3. **Bigger listen backlog** (128 → 1024) so a flood of SYNs doesn't crowd legit ones out of the accept queue.

After the fix: same attack, legit client **88/88 `200 OK`**, worst latency 0.03 s. Up to 1,000 attackers the result is the same. The forced 503 path now delivers the full response (`...\r\n\r\n503 Service Unavailable`) with no reset.

Resolved on the Linux VM (`seedvm2026`, Ubuntu 22.04). With the listen backlog at 1024 and the kernel's `somaxconn` at 4096, the same 1,500-attacker slowloris gives the legit client **88/88 `200 OK`, worst latency 0.03 s** — identical to the 600 case. So the macOS 1,500 stall was an OS resource ceiling, not a server flaw: macOS caps the accept queue at 128 (`kern.ipc.somaxconn`) regardless of what `listen()` requests.

The VM also exposed a trap worth recording: the *first* 1,500 run failed (3/88, then `libgcc_s.so.1 must be installed for pthread_exit` and a core dump) — but that was the **attack harness** running out of file descriptors, not the server. `floods.py` opens 1,500 client sockets in a single process and Ubuntu's default `ulimit -n` is 1024; raising it to 8192 (which `floods.py`'s own docstring instructs) fixed it and the server

passed. The server never needs the raise: `connection_limit()` lifts its own soft fd limit to 4096 and caps concurrency at 500, so it only ever holds ~500 descriptors. Thread-per-connection still spends one fd per slot, so a `selectors` event loop remains the principled fix for pushing the cap far beyond 500.

One defense, and what breaks without it

If the whole-request deadline in `Connection.fill()` were replaced by a per-`recv()` timeout, Slowloris would succeed: a client that sends one byte every few seconds never trips a per-`recv()` timeout, so it can hold a thread forever.